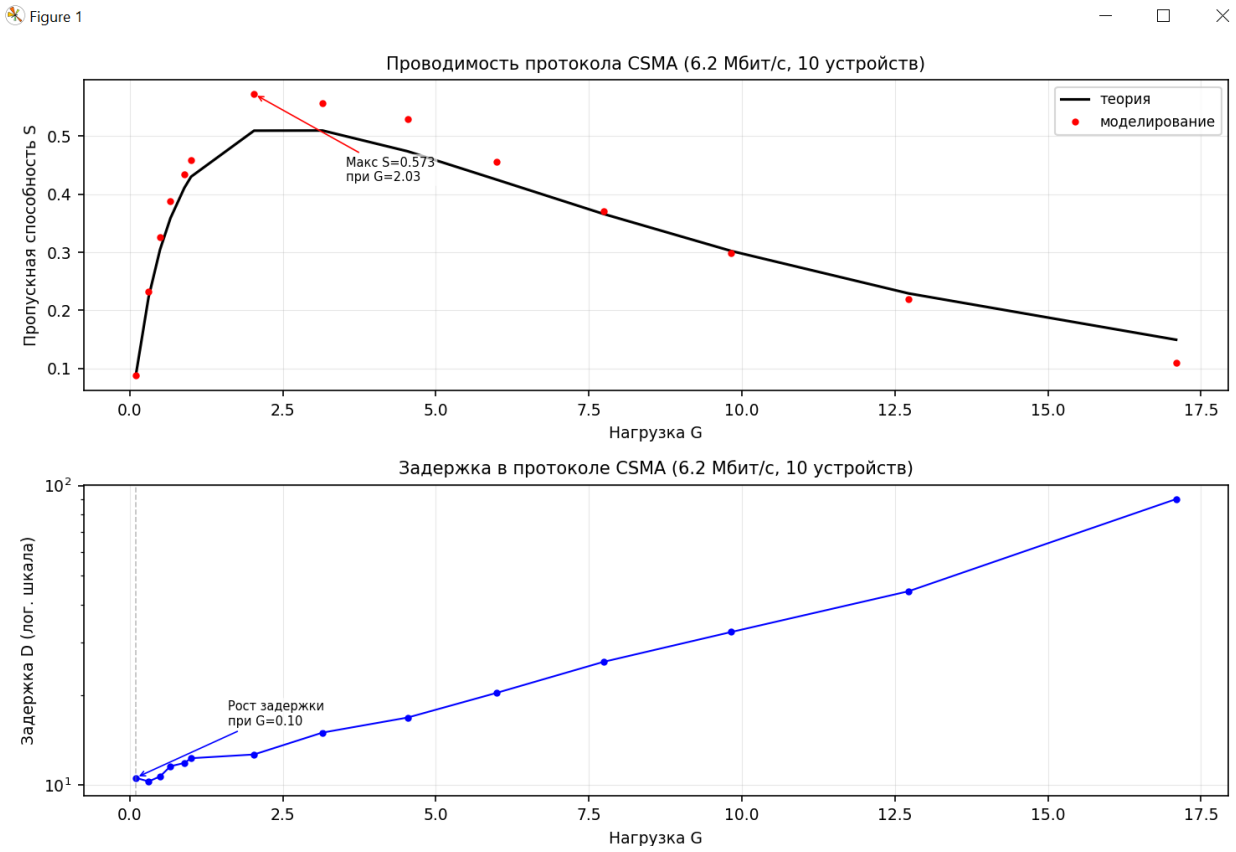
1. Ход выполнения работы



2. Листинг кода

```python
import numpy as np
import matplotlib.pyplot as plt

# Параметры модели
STANDBY = 0       # состояние дежурного приема
TRANSMIT = 1      # состояние передачи
COLLISION = 2     # состояние коллизии
TOTAL = 1000      # общее количество кадров для передачи

# Твои параметры: 50 Мбит/с и 10 устройств
Brate = 6.25e6    # символьная скорость (симв/с) - для 50 Мбит/с при 8 бит/символ
Plen = 500        # длина кадра (симв) - примерно 500 байт
Time = Plen / Brate  # время передачи кадра (с)
Dtime = 0.1       # задержка распространения кадра (% от Time)
delay = Dtime * Time  # задержка распространения кадра (с)
Mnum = 10         # число станций

# Сгенерированный трафик/нагрузка (ограничим сверху, чтобы избежать деления на
ноль)
G = np.concatenate([np.arange(0.1, 1.0, 0.2), np.arange(1, 9.9, 1)])
```

```
# Массивы для хранения результатов
Traffic = np.zeros(len(G))
S = np.zeros(len(G))
Delay = np.zeros(len(G))

print("Начинаю моделирование... Это может занять некоторое время.")
print(f"Параметры: {Mnum} станций, скорость {Brate/1e6:.1f} Мбит/с, длина кадра
{Plen} символов")

for indx, g_val in enumerate(G):
    # Защита от деления на ноль
    if g_val / Mnum >= 1:
        print(f"Пропускаю G={g_val}: превышена максимальная нагрузка")
        continue

    # Интервал генерирования кадров
    Tint = -Time / np.log(1 - g_val / Mnum)
    Rint = Tint

    # Счетчики статистики
    Spnum = 0       # число успешно переданных кадров
    Splen = 0       # объем успешно переданных кадров (симв)
    Tplen = 0       # объем сгенерированных кадров (симв)
    Wtime = 0       # задержка передачи сгенерированных кадров (с)

    # Инициализация
    mgtime = -Tint * np.log(1 - np.random.rand(Mnum))  # время генерирования
кадров
    mtime = mgtime.copy()        # время текущего события
    Mstate = np.zeros(Mnum, dtype=int)      # состояния станций
    Mplen = np.full(Mnum, Plen) # длина кадра
    now_time = np.min(mtime)     # текущее время
    Mstime = np.zeros(Mnum)      # время передачи кадров

    while Spnum < TOTAL:
        # Нахождение станций, завершающих передачу
        idx = np.where((np.abs(mtime - now_time) < 1e-10) & (Mstate ==
TRANSMIT))[0]
        if len(idx) > 0:
            for i in idx:
                Spnum += 1
                Splen += Mplen[i]
                Wtime += now_time - mgtime[i]
                Mstate[i] = STANDBY
                mgtime[i] = now_time - Tint * np.log(1 - np.random.rand())
                mtime[i] = mgtime[i]

        # Нахождение станций в состоянии коллизии
        idx = np.where((np.abs(mtime - now_time) < 1e-10) & (Mstate ==
COLLISION))[0]
        if len(idx) > 0:
```

```
            Mstate[idx] = STANDBY
            # Защита от rand = 1
            rand_vals = np.random.rand(len(idx))
            rand_vals = np.clip(rand_vals, 1e-10, 1 - 1e-10)
            mtime[idx] = now_time - Rint * np.log(1 - rand_vals)

        # Нахождение станций в состоянии ожидания
        idx = np.where((np.abs(mtime - now_time) < 1e-10) & (Mstate ==
STANDBY))[0]
        if len(idx) > 0:
            Tplen += np.sum(Mplen[idx])
            for jj in idx:
                # Проверка, занята ли среда
                busy = False
                for k in range(Mnum):
                    if (Mstate[k] == TRANSMIT or Mstate[k] == COLLISION) and \
                       Mstime[k] + delay <= now_time <= Mstime[k] + delay + Time:
                        busy = True
                        break

                if not busy:
                    Mstate[jj] = TRANSMIT
                    Mstime[jj] = now_time
                    mtime[jj] = now_time + Mplen[jj] / Brate
                else:
                    # Защита от rand = 1
                    rand_val = np.random.rand()
                    rand_val = np.clip(rand_val, 1e-10, 1 - 1e-10)
                    mtime[jj] = now_time - Rint * np.log(1 - rand_val)

        # Нахождение станций в состоянии передачи или коллизии
        transmit_mask = (Mstate == TRANSMIT) | (Mstate == COLLISION)
        if np.sum(transmit_mask) > 1:
            Mstate[transmit_mask] = COLLISION

        # Обновляем текущее время
        now_time = np.min(mtime)

    # Сохранение результатов
    Traffic[indx] = Tplen / Brate / now_time
    S[indx] = Splen / Brate / now_time
    Delay[indx] = Wtime / TOTAL * Brate / Plen

    # Вывод прогресса
    print(f"Обработано {indx+1}/{len(G)} значений G (G={g_val:.2f})")

# Убираем нулевые значения (если были пропущены)
valid_indices = Traffic > 0
Traffic = Traffic[valid_indices]
S = S[valid_indices]
Delay = Delay[valid_indices]
```

```
G = G[valid_indices]

# Теоретическая пропускная способность для протокола CSMA
theory = Traffic * np.exp(-Dtime * Traffic) / (Traffic * (1 + 2 * Dtime) +
np.exp(-Dtime * Traffic))

# УМЕНЬШАЕМ ГРАФИКИ И НАДПИСИ
plt.rcParams.update({'font.size': 9})  # уменьшаем базовый размер шрифта
plt.figure(figsize=(10, 7))  # уменьшаем размер фигуры

# График проводимости
plt.subplot(2, 1, 1)
plt.plot(Traffic, theory, 'k-', linewidth=1.5, label='теория')
plt.plot(Traffic, S, 'ro', markersize=3, label='моделирование')
plt.grid(True, alpha=0.3, linewidth=0.5)
plt.xlabel('Нагрузка G', fontsize=9)
plt.ylabel('Пропускная способность S', fontsize=9)
plt.legend(fontsize=8, loc='upper right')
plt.title(f'Проводимость протокола CSMA ({Brate/1e6:.1f} Мбит/с, {Mnum}
устройств)', fontsize=10)

# Добавим аннотации с точками перегиба (уменьшенные)
max_s_idx = np.argmax(S)
plt.annotate(f'Макс S={S[max_s_idx]:.3f}\nпри G={Traffic[max_s_idx]:.2f}',
             xy=(Traffic[max_s_idx], S[max_s_idx]),
             xytext=(Traffic[max_s_idx]+1.5, S[max_s_idx]-0.15),
             arrowprops=dict(arrowstyle='->', color='red', lw=0.8),
             fontsize=7, bbox=dict(boxstyle='round,pad=0.2', fc='white',
alpha=0.7, ec='none'))

# График задержки
plt.subplot(2, 1, 2)
plt.semilogy(Traffic, Delay, 'bo-', linewidth=1, markersize=3)
plt.grid(True, alpha=0.3, linewidth=0.5)
plt.xlabel('Нагрузка G', fontsize=9)
plt.ylabel('Задержка D (лог. шкала)', fontsize=9)
plt.title(f'Задержка в протоколе CSMA ({Brate/1e6:.1f} Мбит/с, {Mnum} устройств)',
fontsize=10)

# Отметим точку, где задержка начинает резко расти (уменьшенная аннотация)
delay_threshold_idx = np.where(Delay > 1)[0]
if len(delay_threshold_idx) > 0:
    first_high_delay = delay_threshold_idx[0]
    plt.axvline(x=Traffic[first_high_delay], color='gray', linestyle='--',
alpha=0.5, linewidth=0.8)
    plt.annotate(f'Рост задержки\nпри G={Traffic[first_high_delay]:.2f}',
                 xy=(Traffic[first_high_delay], Delay[first_high_delay]),
                 xytext=(Traffic[first_high_delay]+1.5,
Delay[first_high_delay]*1.5),
                 arrowprops=dict(arrowstyle='->', color='blue', lw=0.8),
```

```
                fontsize=7, bbox=dict(boxstyle='round,pad=0.2', fc='white',
alpha=0.7, ec='none'))

plt.tight_layout(pad=1.5)  # уменьшаем отступы
plt.show()

# УМЕНЬШАЕМ ТАБЛИЦУ С РЕЗУЛЬТАТАМИ
print("\n" + "="*50)
print("РЕЗУЛЬТАТЫ МОДЕЛИРОВАНИЯ")
print("="*50)
print(f"{'G':>4} {'S теор':>7} {'S мод':>7} {'Откл':>6} {'D':>6}")
print("-"*50)
for i in range(len(G)):
    deviation = abs(S[i] - theory[i]) / theory[i] * 100 if theory[i] > 0 else 0
    print(f"{G[i]:4.1f} {theory[i]:7.4f} {S[i]:7.4f} {deviation:5.1f}%
{Delay[i]:6.2f}")

print("\n" + "="*50)
print("АНАЛИЗ РЕЗУЛЬТАТОВ")
print("="*50)
print(f"Макс S: {S[max_s_idx]:.4f} при G={Traffic[max_s_idx]:.2f}")
print(f"Среднее отклонение: {np.mean(np.abs((S - theory)/theory)):.2%}")

print("\nРЕКОМЕНДАЦИИ:")
print("-"*30)
if len(Traffic[Traffic < 1]) > 0:
    print(f"• Легкая G<1: {np.mean(S[Traffic < 1])*Brate/1e6:.1f} Мбит/с")
if len(Traffic[(Traffic >= 1) & (Traffic < 3)]) > 0:
    print(f"• Средняя 1-3: S={np.mean(S[(Traffic >= 1) & (Traffic < 3)]):.2f}")
if len(Traffic[Traffic >= 3]) > 0:
    print(f"• Высокая G>3: S={np.mean(S[Traffic >= 3]):.2f},
D={np.mean(Delay[Traffic >= 3]):.1f}")
```

3. Контрольные вопросы

1. **pr-CSMA (persistent CSMA)** — настойчивый CSMA работает следующим образом:

- **Основная особенность:** Станция, имеющая кадр для передачи, непрерывно прослушивает среду и передает сразу, как только среда освобождается (переходит в состояние IDLE).
- **Вероятность передачи:** В отличие от np-CSMA (ненастойчивого), где станция ждет случайное время, в pr-CSMA станция передает с вероятностью 1 при обнаружении свободной среды.
- **Высокий риск коллизий:** Если несколько станций ждали освобождения среды, они начнут передавать одновременно, что приведет к коллизии.

- **Вариант p-CSMA:** Существует также p-настойчивый CSMA, где станция передает с вероятностью p, а с вероятностью (1-p) ждет следующего слота.

2. Временная диаграмма функционирования протокола pr-CSMA

У нас есть две станции (Станция 1 и Станция 2) и ось **Нормированного времени** для каждой из них. На этой оси отмечены ключевые события и интервалы, из которых складывается полный цикл работы протокола.

- **B (Busy - Занято):** Это промежуток времени, когда среда занята передачей. На диаграмме это самый длинный блок.
- **I (Idle - Простой):** Это промежуток времени, когда среда свободна и никто не передает.
- **Y и Y_1:** Это **случайное время ожидания**. Оно возникает из-за коллизий. Когда две станции начинают говорить одновременно, они создают "кашу" (коллизию). Y — это время от начала этой каши до момента, когда обе станции понимают, что произошла коллизия, и замолкают. Y_1 — это время, которое прошло с начала цикла до момента, когда вторая станция "вклинилась" в разговор первой.
- **α (альфа):** Это **задержка распространения** (та самая `Dtime` в коде). Это время, за которое сигнал доходит от одной станции до другой. Пока сигнал летит, другая станция может не знать, что среда уже занята, и тоже начать передачу, что и приводит к Y.
- **a:** Это, скорее всего, та же самая задержка распространения α, просто обозначенная другой буквой.
- **Цикл:** Это полный период работы протокола, который состоит из фазы "Занято" (B) и фазы "Простой" (I). После коллизии или успешной передачи всё начинается заново.

3. Связь B, I, U с пропускной способностью S:

Для протокола pr-CSMA (и CSMA в целом) пропускная способность определяется формулой:

S = U / (B + I)

Где:

- **U** — нормированная продолжительность успешной передачи кадров (без коллизий)
- **B** — нормированная продолжительность состояния BUSY (среда занята)
- **I** — нормированная продолжительность состояния IDLE (среда свободна)

Физический смысл: Пропускная способность — это доля времени, когда среда действительно передает полезные данные, от общего времени работы (занятость + простой).

4. Связь нагрузки G с продолжительностью успешной передачи U

U = G · e^(-αG)

Где:

- **G** — нормированная нагрузка (среднее количество кадров, генерируемых за время передачи одного кадра)
- **α** — нормированная задержка распространения (в долях от времени передачи кадра)
- **e^(-αG)** — вероятность того, что за время распространения сигнала (α) ни одна другая станция не начнет передачу

Пояснение: Чем выше нагрузка G, тем больше кадров генерируется, но тем выше вероятность коллизий, поэтому U растет только до определенного предела, а затем падает.

5. Связь продолжительности IDLE с нагрузкой G:

I = 1/G

Пояснение:

- В состоянии IDLE кадры не генерируются
- При низкой нагрузке (G мало) простои длинные
- При высокой нагрузке (G велико) простои короткие
- Это следует из пуассоновского характера поступления кадров — средний интервал между кадрами обратно пропорционален интенсивности

6. Вероятностный характер параметра B (продолжительность BUSY):

Нормированная продолжительность состояния BUSY определяется выражением:

B = 1 + α + Y

Вероятностный характер связан с **Y** — случайной величиной, которая представляет собой:

- **Y** — время от момента возникновения коллизии до момента, когда все станции обнаруживают коллизию и прекращают передачу
- Распределение Y определяется вероятностью того, что в интервале (α - y) не будет сгенерировано ни одного кадра:

F_Y(y) = e^[-(α - y)G], где y ≤ α

Среднее значение Y:
Ȳ = α - (1/G)(1 - e^(-αG))

Почему это вероятностно:

1. Моменты генерации кадров случайны (пуассоновский процесс)
2. Моменты возникновения коллизий случайны
3. Количество станций, участвующих в коллизии, случайно
4. Время "перекрытия" кадров зависит от случайного момента начала передачи второй станции